Character-Level Text Prediction with Recurrent Neural Networks Machines Attempt to Overthrow Shakespeare

Jorge Rodríguez Toledo

Institute for Cross-Disciplinary Physics and Complex Systems (IFISC)
Postgraduate Centre of the University of the Ballearic Islands, 07122 Palma de Mallorca, Spain

May 26, 2026

ABSTRACT

This work trains three different recurrent architectures, a Simple RNN, a GRU, and a two-layer LSTM, on Shakespeare's collected works to test their ability to predict the next character in a sequence. We changed the type of recurrent cell but kept the same embedding, hidden size, and dropout settings. Each model was trained in two configurations: one including a self-attention layer, and one without (baseline). Training ran until validation loss plateaued under a custom early-stopping rule with a 10 % held-out split. Despite expectations, adding self-attention layers to the three architectures failed to improve results as the original baseline models actually performed better. The best overall model was the LSTM without attention, achieving a validation loss of 1.528 and an accuracy of 52.7 %. The Simple RNN was the most affected by attention, suffering a the largest validation loss increase. Beyond these comparisons, we explored how temperature shapes the LSTM's generated text, examined the linguistic patterns each model learns, and broke down an example failure case. Ultimately, this research shows recurrent models can master long-range dependencies, transforming raw character sequences into coherent structures for modeling the dynamics of complex informational environments.

I. INTRODUCTION

Teaching a machine to write like Shakespeare is an idea that appears misleadingly simple at first that turns out to reveal a great deal about how neural networks learn language. At its core, the task is: given the characters seen so far, predict the next one. Do that well enough, and coherent words, sentences, and even dramatic dialogue begin to emerge.

Letters don't mean much on their own, so the network needs a "memory" to keep track of where it's been and make sense of what's coming next. That's where recurrence comes in. The magic is in the cell design; it's the difference between a model that maintains a narrative thread and one that forgets how its own sentence started.

Three families of recurrent cell are compared in this study. The **Vanilla RNN** [2][7] updates its hidden state as:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t),$$

A simple nonlinear recurrence that struggles to propagate gradients over long sequences.

The **GRU** [3] introduces an update gate z_t and a reset gate r_t:

$$\begin{aligned} r_t &= \sigma(W_{xr}\,x_t + W_{hr}\,h_{t-1} + b_r) \\ z_t &= \sigma(W_{xz}\,x_t + W_{hz}\,h_{t-1} + b_z) \\ \tilde{h}_t &= \tanh\big(W_{xh}\,x_t + W_{hh}(r_t \odot h_{t-1}) + b_h\big) \\ h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t \end{aligned}$$

The **LSTM** [4] adds a separate cell state c_t controlled by forget, input and output gates:

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$\begin{aligned} c_t &= f \odot c_{t-1} + i \odot g \\ h_t &= o \odot \tanh(c_t) \end{aligned}$$

The cell-state pathway provides an easier route for the model to learn long-distance dependencies.

This report walks through how all three models were built, trained and evaluated, and analyses what the results tell us about the trade-offs between simplicity, capacity and convergence speed.

II. DATA PREPROCESSING

We train on the *Tiny Shakespeare* dataset [6] that contains around one million characters. All 65 unique characters are sorted and assigned integer indices. Two lookup tables handle the conversion from characters to integers and viceversa.

The encoded text is divided into a 90/10 split before windowing, so that no window crosses the split boundary. Overlapping sliding windows of $L = 100$ characters with step $s = 33$ are extracted from each split, yielding ≈30,000 training sequences and ≈3,300 validation sequences. Each window of length $L + 1$ is split into an input of length L and a target shifted one position to the right, so that at every time step the model must predict the next character:

Input	`"To be or not to be, that is the qu"`
Target	`"o be or not to be, that is the que"`

Using a held-out validation set is essential as it gives early stopping an unbiased signal and ensures the reported metrics reflect generalisation, not overfitting.

Preprocessing details can be found in Appendix B.

III. MODEL ARCHITECTURES

Each of the three cell types is evaluated in two configurations, yielding six models in total. The **attention** variant follows the full pipeline: an embedding layer (d=64), two stacked recurrent layers (128 units each) separated by 20 % dropout, a self-attention layer and finally a dense output with 65 logits (one per character in the vocabulary). The **baseline** variant is identical but with the attention layer omitted. The difference in performance depends entirely on the cell choice and wether the model presents or not self attention.

The self-attention layer lets every part of the sequence "see" every other part. This prevents the model from losing track of earlier information over time. Attention scores were scaled to ensure stability during runs.

Table 1: Number of parameters per variant.

Model	Baseline	Attention
Simple RNN	70,145	70,146
GRU	186,113	186,114
LSTM	242,945	242,946

Attention adds only one trainable parameter, so any performance gap we observe is purely a result of the architecture itself. It is a true test of how well each design can actually move information through a sequence, rather than just a contest of which model has more raw parameters.

Model architecture details can be found in Appendix C.

IV. TRAINING

Each model is compiled with the **Adam optimiser** with a learning rate $\eta = 10^{-3}$, the standard choice for deep learning. The loss function is **Sparse Categorical Cross-Entropy**. To maintain the optimal checkpoint and prevent redundant computation, we introduced three callbacks:

- **ModelCheckpoint** writes the best weights whenever validation loss improves.
- **EarlyStopping** monitors validation loss with a patience of 8 epochs, restoring the best weights when training stops.
- **ReduceLROnPlateau** halves the learning rate after 3 non-improving epochs, down to 10^{-5}. This allows the optimiser to continue making progress once the initial rapid descent slows.

We fixed `max_epochs=150` as a safety net, but early stopping is expected to trigger well before that limit. After each variant finishes, the full metrics, total epochs reached, and training time are saved. This allows the loss curves to be reproduced without rerunning training.

V. TEXT GENERATION

A. Generation Algorithm

Before generating a new text, we build fresh stateless inference model with the same architecture as the trained model. Then the learned weights are loaded from the trained weight files.

The `generate_text` function implements autoregressive generation [6]. First, the seed string is encoded into an integer list forming the initial context. At each generation step, the complete context is passed to the model as a tensor of shape $(1 \times \ell)$, where ℓ increases by one with each new character. The model returns a logit matrix where we only keep the last position that encodes the prediction for the next character. Temperature scaling is then applied and the logits are shifted by their maximum value

to prevent numerical overflow before exponentiation. The sampled character is appended to both the output string and the input list, extending the context for the next step.

B. Temperature

The temperature T controls the sharpness of the sampling distribution: $T < 1$ is safer and more conservative, $T = 1$ is balanced, and $T > 1$ is more varied and noisier.

Table 2: Effect of temperature on the distributions.

Temperature (T)	Distribution Effect
$T < 1$	Sharpened
$T = 1$	Unchanged
$T > 1$	Flattened

VI. RESULTS AND DISCUSSION

A. Training Curves

The final convergence metrics are presented in table 3. Figure 1 shows the validation loss and accuracy evolution for all six model variants. Three phases characterise the shared learning dynamics: a rapid initial descent as all models absorb character-frequency statistics and bigrams, a middle phase where gating becomes decisive, and a final plateau where early stopping fires. For all baseline variants, the curves consistently sit at or below their attention counterparts, confirming that adding the attention layer does not help any architecture. Among cell types, the LSTM reaches the best loss in both variants, the GRU follows with a noisier trajectory and the Simple RNN runs longest yet finishes highest in both variants. This is a clear sign that extra epochs do not substitute for a better cell design.

Table 3: Convergence metrics for all six variants.

Model	Variant	Ep.	Val Loss	Time
RNN	Baseline	81	1.6589	7.0 min
RNN	Attention	74	1.7702	6.3 min
GRU	Baseline	49	1.5340	4.1 min
GRU	Attention	41	1.5626	3.5 min
LSTM	Baseline	70	1.5283	5.6 min
LSTM	Attention	63	1.5419	5.2 min

In figure 1 there is a sharp, discontinuous upward jumps in the Simple RNN's validation loss, present in milder form in the GRU and absent in the LSTM. Two mechanisms interact to produce them. When the `ReduceLROnPlateau` callback halves the learning rate, it temporarily miscalibrates Adam's moment estimates, causing a brief jump in loss before the descent stabilizes. In ungated cells architectures like the Simple RNN, this effect is amplified because the model lacks a protected cell state. Since the RNN's hidden state is entirely overwritten at every step, any gradient fluctuation propagates directly into the predictions. While the GRU's update gate z_t provides partial insulation, it remains vulnerable if the gate saturates during a full time step. The LSTM, by contrast, is structurally shielded. For a gradient excursion to disrupt the cell state c_t, the forget gate f and input gate i must saturate simultaneously. This is statistically very unlikely and so ensures the training curve is smooth from start to finish.

The full rankings are shown in table 4 alongside two naive baselines. The best performing model is the LSTM with no attention, achieving an accuracy of 52.7 % and a loss of 1.528. Figure 2 shows this model's evolution in detail. The training and validation curves drop smoothly and steadily from start to finish without any of the erratic spikes or setbacks that usually signal a model is struggling to learn. Similarly, in the accuracy curves also show a steady increase indicating the model is learning correctly.

Table 4: Ranking of all models alongside two naive baselines.

Model	Val Loss	Val Acc
Uniform random	4.174	1.54 %
Unigram baseline	3.309	15.3 %
Simple RNN (attention)	1.770	48.3 %
Simple RNN (baseline)	1.659	51.0 %
GRU (attention)	1.563	51.3 %
GRU (baseline)	1.534	52.6 %
LSTM (attention)	1.542	52.4 %
LSTM (baseline)	**1.528**	**52.7** %

B. The Attention Paradox

Naively one would think that adding an extra layer of complexity to the model's architecture should improve its performance. However, figure 3 shows that adding self-attention degraded every architecture. The Simple RNN sustained the greatest losses dropping to an accuracy of 48.3 % that positions RNN-with-attention as the worst-performing model. Even the GRU and LSTM declined in performance, though the LSTM's "cell-state highway" helped it withstand the damage better than the others. We can also check

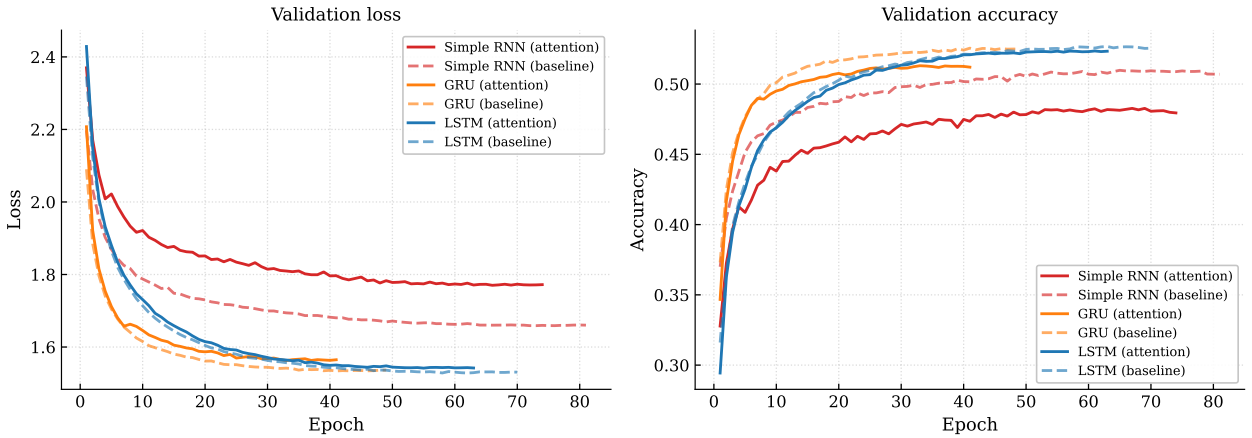

Figure 1: Validation loss (left) and accuracy (right) for all six model variants.

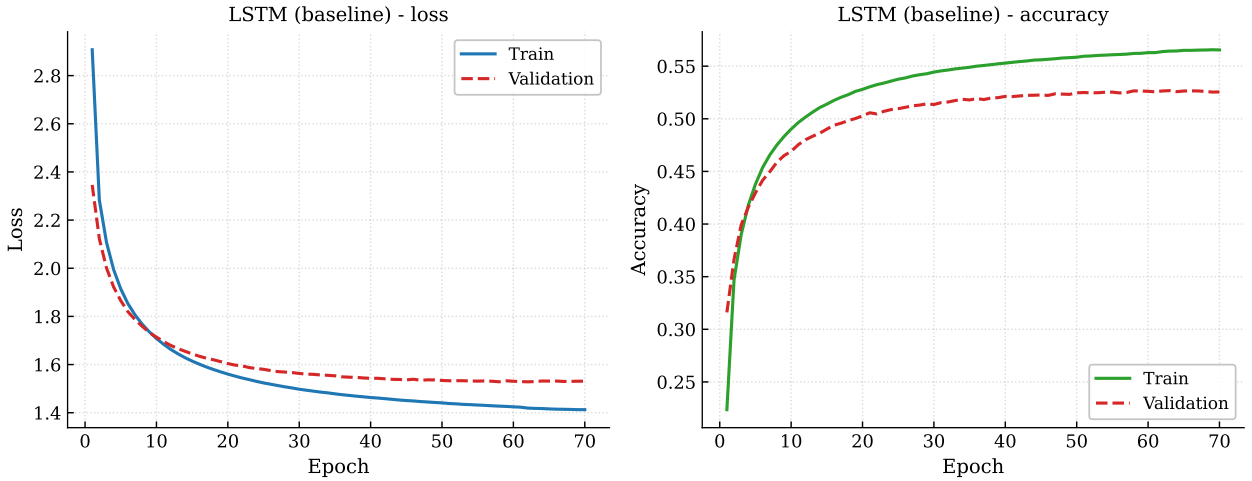

Figure 2: Train vs. validation loss (left) and accuracy (right) for LSTM without attention, the best overall model.

that every attention variant also converged in less epochs. This is not a sign of efficiency but of premature plateau. Noisy attention gradients triggered early stopping before the model could fully converge.

The failure of the attention mechanism comes from two compounding effects. First, the layer lacks the necessary projection matrices, relying on a simplified $Q{=}K{=}V{=}x$ setup with a single learned scalar. Without dedicated W_Q, W_K, W_V matrices, the attention weights are determined by raw dot-product similarity between hidden states. This metric carries little structural meaning when processing text at character level. The mechanism cannot learn to filter out irrelevant positions, it simply produces a globally averaged mixture of the recurrent cell's output.

Second, the value of that mixture depends entirely on the stability of the encoded hidden states. The LSTM's cell-state highway preserves long-range context in a relatively stable form. In contrast, the Simple RNN's hidden states degrade rapidly due to vanishing gradients. By the midpoint of a 100-step sequence, earlier characters are not very informative, so attending over them introduces noise into the signal. This establishes a clear inverse relationship: the poorer the network's memory retention, the more disruptive projection-free attention becomes, a result that is quite the opposite of what standard intuition might suggest.

C. Generative Quality

For a further model comparison we ran all three baseline variants using same seed `"To be"` and temperature $T{=}0.8$ for a sequence of 100 characters. The outputs mirrors the loss ranking exactly:

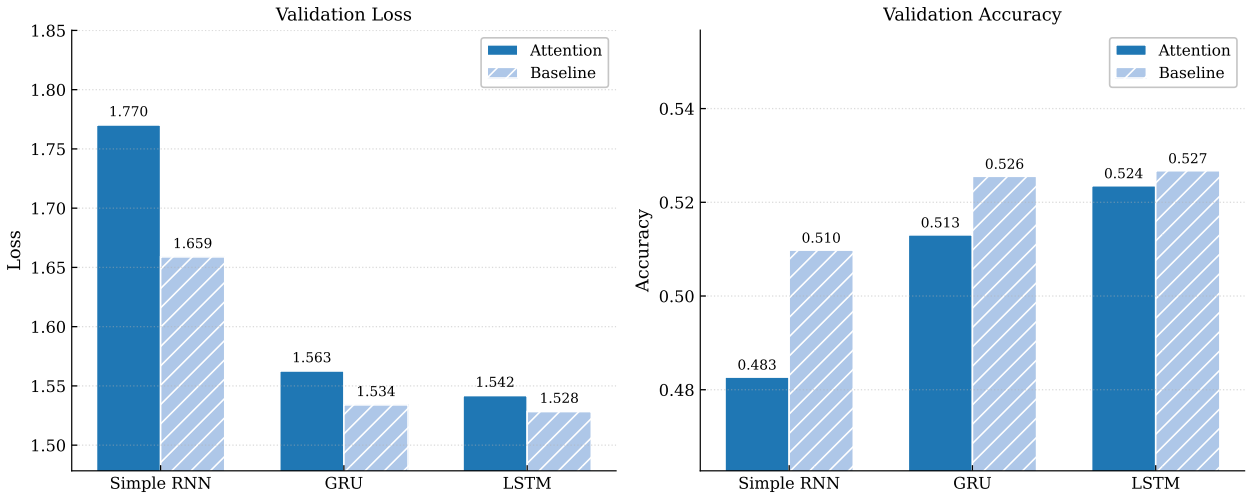

Figure 3: Validation loss (left) and accuracy (right) for all six variants.

Simple RNN (baseline):

```
To be so many much liege?

BENVOLIO:
But who and the ramrer benore we shall,
Were gentles of never gast;
```

GRU (baseline):

```
To be my lord,
Both at my lord,
My father's winf and injuries,
I then which for death comes shall See,
un
```

LSTM (baseline):

```
To be crown.

LADY CAPULET:
My lord.
Grown, why, thy gracious father was
enough.

KING EDWARD IV:
Thou f
```

We find three levels of generative quality. All models handle common short words and punctuation, but the Simple RNN still produces invalid character sequences ("ramrer", "benore") and its sole speaker cue (`BENVOLIO:`) is immediately undermined by garbled output. The GRU advances to mostly real words and correct punctuation, yet its internal context begins to recycle. The words "my lord" appears twice in rapid succession and it fails to to produce a speaker transition. Only the LSTM masters the playwright's conventions: two properly formatted speaker cues (`LADY CAPULET:`, `KING EDWARD IV:`) each followed by coherent Shakespearean prose. The shared ceiling across all three models is the 100-character training window: both the GRU and LSTM samples end mid-word ("un", "Thou f"), confirming that no model reliably represents structure beyond that limit.

To probe the LSTM further, we ran a longer sequence of 200 characters using the seed `"ROMEO:"` at temperature T=1.0:

LSTM (baseline):

```
ROMEO:
It gone as it do begels endation to
make him not; tell my tremblan of an
over his
omprise.  What loting it; deveramed than
Fot, my lord.

DUKE VINCENTIO:
What, this is your will; and this so I
can spe
```

We notice the model correctly introduces a second speaker mid-sequence (`DUKE VINCENTIO:`) with the proper format, demonstrating that the LSTM has internalised the play's structural convention of alternating voices. By raising the temperature from T=0.8 to T=1.0, it directly increases the lexical noise. Non-words such as "begels" and "deveramed" appear alongside Shakespearean phrasing like "my lord" and "this is your will". This is consistent with the flatter sampling distribution at T=1.0, which raises the probability of low-frequency characters breaking some of the coherent patterns in the process. As expected from the 100-character training window, the sample ends mid-word ("spe"), a reminder that reliable long-range context is the hard ceiling for these models.

We also analyse a failure example from the baseline LSTM with the seed `"zzz"` and temperature T=2.0:

```
zzzay;.-Aater yea, &hay
o' Stiffer Iscuess tozel year.  Seak!
Mefissihe!  his pains ban Arskun'd,
Ismend
```

The trigram `zzz` is absent from the training sequences, so the hidden state is initialised in a region of activation space it had never visited before. This instability is compounded when the logits are divided by $T = 2.0$: the distribution is flattened so severely that low-probability characters become near-equally likel as any other. As a result, each sampled character pushes the state further away from the known training distribution. This is a case of compounding divergence known as exposure bias [1].

T	Character of output
0.3	Repetitive loops with common phrases recycled endlessly.
0.6	More varied, mostly real words but still conservative.
0.8	Shows the best balance between fluency and Shakespearean phrasing.
1.0	Slightly noisier with some occasional invented words.
1.2	Noticeably noisier with structural drift and invented character sequences.
1.5	Frequent nonsense and the structure fully collapses.

Table 5: Qualitative effect of temperature T on LSTM output (seed `"HAMLET:"`).

Table 5 summarises the LSTM's output quality across six temperature values. For low T diversity is traded for safety, ultimately collapsing into repetitive loops. On the other hand, high T prioritizes coherence for variety, quickly dissolving into nonsense. The optimum is $T = 0.8$, which presents correct punctuation, recognisable Shakespearean vocabulary, and enough stochasticity to avoid repetition.

The choice of seed introduces a distinct thematic dimension. The seed `"To be"` triggers reflective monologue whereas `"ROMEO:"` activates the emotional register of a typical early-play dialogue. However, this contextual influence remains temporary. After some characters, the initial seed is forgotten as the hidden state becomes diluted by the model's own predictions.

D. Future Directions

Based on the failure modes identified earlier, two extensions can be considered. First, transitioning to **Multi-head attention with learned projections** (W_Q, W_K, W_V matrices) [8] would provide the expressive power the attention mechanism currently lacks. This is particularly promising for the GRU and LSTM, as their hidden states are sufficiently structured to benefit from being mapped into specialized subspaces.

Second, refining the generation algorithm with **nucleus (top-p) sampling** [5] would directly mitigate repetitive collapse by dynamically restricting the sampling vocabulary to the most probable candidates. It can be complemented with **scheduled sampling** [1] that directly addresses exposure bias by training the model to recover from its own mistakes. Both additions represent possible future implementations that may resolve the projects greatest flaws.

Finally, increasing the size of the hidden layers or stacking additional recurrent layers would be expected to improve performance further, as larger representations allow the models to capture more complex long-range patterns. However, this comes at the cost of significantly longer training times and greater hardware requirements, making such improvements contingent on access to more powerful computing resources.

VII. CONCLUSION

The central finding of this study is that projection-free self-attention does not improve character-level recurrent models. Every baseline model outperformed its "upgraded" counterpart where the Simple RNN experienced the worse decline in performance. It's a clear sign that attention without proper structure is more of a distraction than a feature.

The LSTM showed exactly why it's the veteran of the group. Its "cell-state highway" kept the training process smooth and stable, avoiding the chaotic performance spikes we saw in the GRU and Simple RNN. This proves that the fundamental cell type is a much more powerful lever for performance than a basic attention layer. To really unlock this, we need to bring back the complexity we cut out: swapping to full multi-head attention with learned W_Q, W_K, and W_V matrices is the only way forward. Combined with smarter sampling techniques, we can fix the coherence issues we hit here and finally get these models to write so naturally that you can't tell the difference between Shakespeare and the code.

REFERENCES

[1] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[2] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks, 1994.

[3] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. arXiv preprint arXiv:1406.1078, 2014.

[4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[5] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.

[6] Andrej Karpathy. The unreasonable effectiveness of recurrent neural networks. Blog post, 2015.

[7] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. International Conference on Machine Learning (ICML), 2013.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Appendix A: Implementation Details

This appendix discusses the implementation choices and the reasons behind each decision. The code is **original** and can be found in the attached file *Chracter-LevelTextPrediction.ipynb*. See the attached *README.md* and *requirements.yml* for reproduction instructions and package dependencies respectively.

The random seeds are fixed to 5721. However, numerical differences between runs are possible due to non-deterministic CUDA operations. The results reported here were obtained from a single run. The model weights for this run can be accessed in the corresponding zip file to produce the numerical results and figures found in this report. However, text generation is inherently stochastic even with a fixed global seed, so two generation runs from the same seed and temperature will produce different outputs.

Appendix B: Preprocessing

The sequence length $L = 100$ characters provides enough context for the model to observe complete words and general patterns. Shorter sequences would prevent the recurrent layers from learning word-order regularities, while longer ones would slow down the sliding-window construction and reduce the number of training examples. The batch size $B = 128$ is large enough for smooth, steady learning and ensuring we make the most of the GPU's memory.

The windows are extracted with a step of $s = 33$ characters, so consecutive windows overlap. This triples the number of gradient updates per epoch without increasing model size or memory cost.

Appendix C: Model Architecture

The embedding dimension $d = 64$ maps each of the 65 vocabulary tokens to a dense continuous vector. A smaller embedding would numerically suffice for a 65-symbol vocabulary, but a larger one gives the recurrent layers richer input representations and facilitates smoother gradient flow back to the embedding table.

The selft-attention layer is placed after the final recurrent dropout block and receives the full output sequence as both query and value ($Q = K = V = x$). It uses scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

with a single learned scalar in place of the fixed $1/\sqrt{d_k}$ factor [8]. This lets every position attend directly to all others, providing a gradient-friendly path to distant context that bypasses the recurrent bottleneck. During training the full sequence is available at once, so each position attends meaningfully to all others.